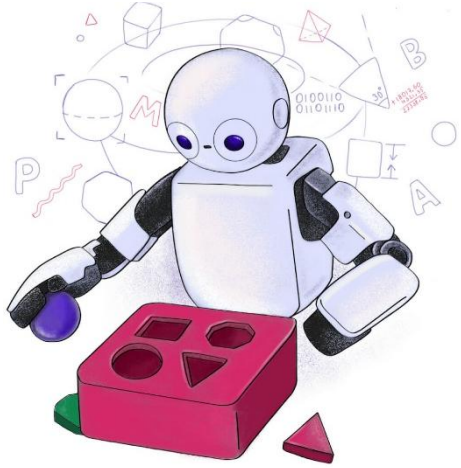
TP558 - Tópicos avançados em Machine Learning:

Transformers



Inatel

Bryan Fernando Sarango Rodriguez
bryan.rodriguez@dtel.Inatel.br

ABERTURA · CONTEXTO DO TEMA

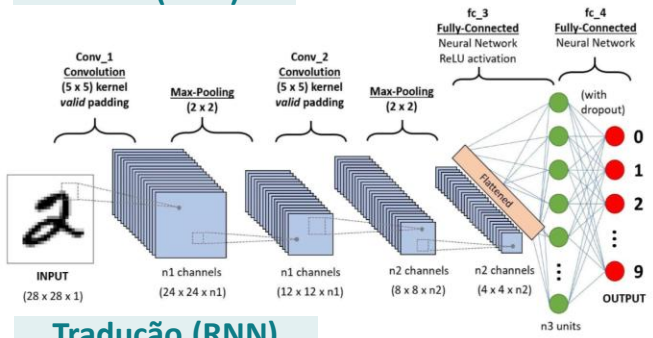
De uma arquitetura por tarefa a uma arquitetura para tudo

Antes de 2017, cada problema tinha seu modelo; depois, o mesmo bloco serve para quase tudo.

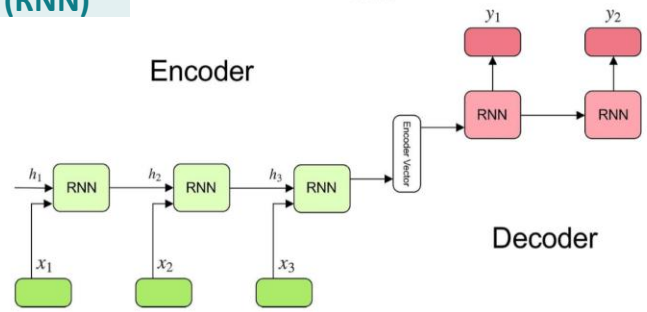
Antes: uma arquitetura por tarefa

Depois: um único bloco

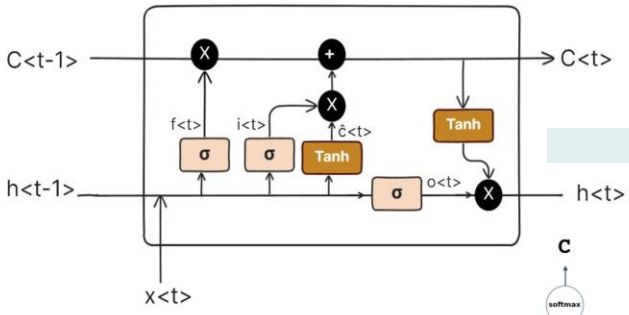
Visão (CNN)



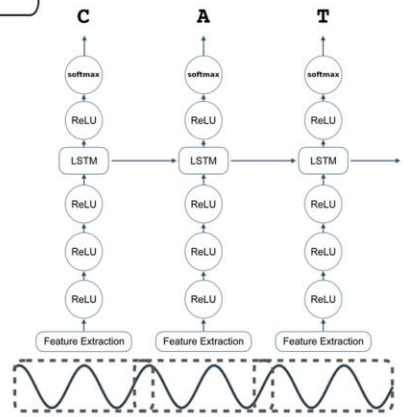
Tradução (RNN)

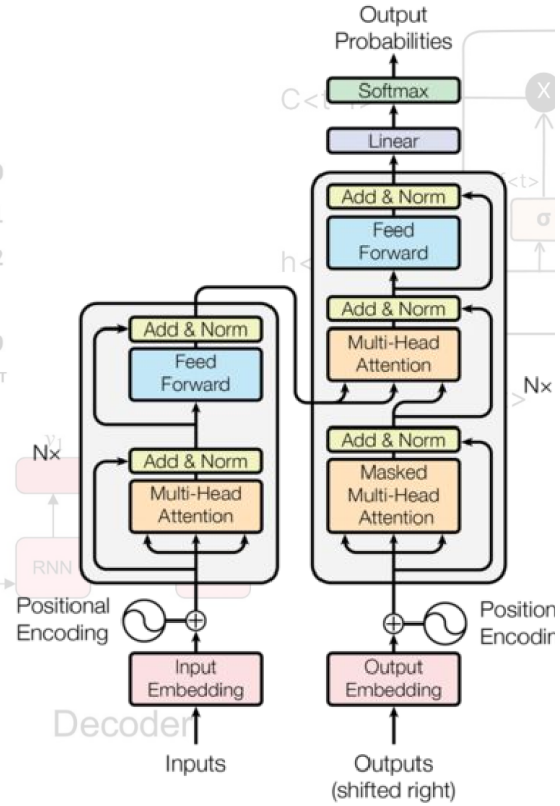


Sequências (LSTM)



Fala →





Hoje, o mesmo bloco aparece em:

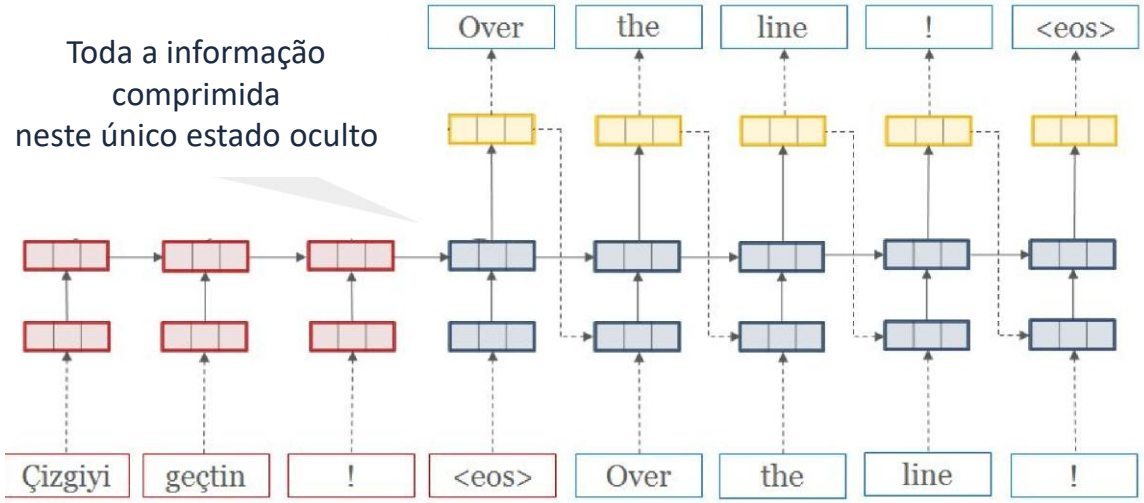
- Tradução
- ChatGPT / GPT
- Busca (BERT)
- Visão (ViT)
- Fala (Whisper)
- Proteínas (AlphaFold)

ABERTURA · PROBLEMA E MOTIVAÇÃO

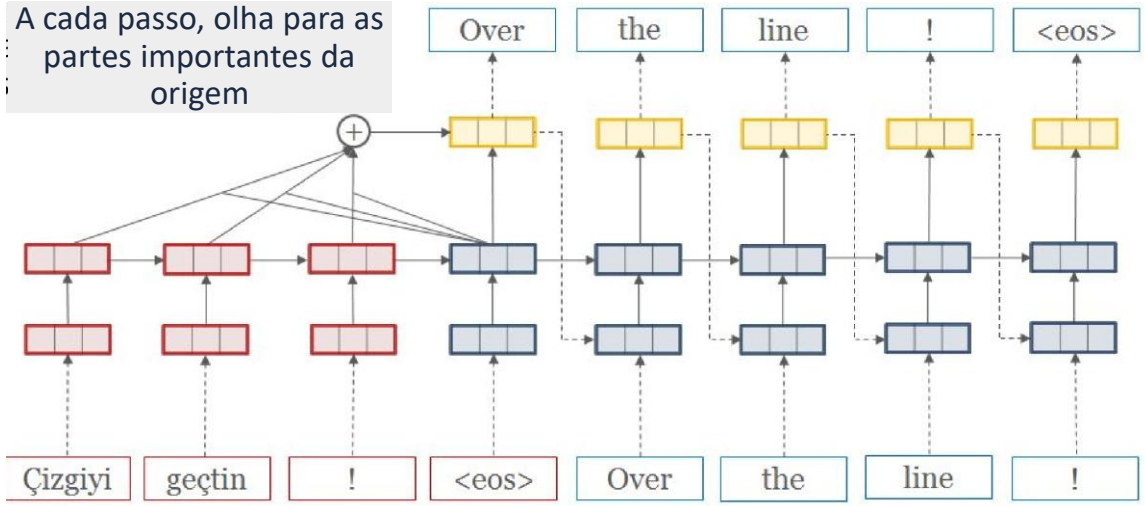
O gargalo das RNNs e a ideia da atenção

Traduzir exige olhar para o contexto certo, mesmo distante; e a RNN processa uma palavra por vez.

2014 · Seq2Seq: tudo num único vetor



2015 · Atenção (Bahdanau et al.)



Exemplo da figura: turco → inglês (“Çizgiyi geçtin!” → “Over the line!”)

Por que o contexto importa?

Sentei no **banco** da **praça**.

→ assento

Depositei **dinheiro** no **banco**.

→ instituição financeira

Mesma palavra, significados diferentes: o modelo precisa relacionar cada palavra com as outras, próximas ou distantes.

INTRODUÇÃO DO ARTIGO

Attention Is All You Need (NeurIPS 2017)

Proposta: trocar a recorrência por atenção, e somente atenção.



Attention is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Autores: Google Brain, Google Research e Univ. de Toronto · arXiv:1706.03762

HIPÓTESE (IMPLÍCITA)

Atenção basta para modelar a relação entre tokens, com tradução igual ou melhor e treino muito mais rápido e paralelo.

1 **Objetivo**

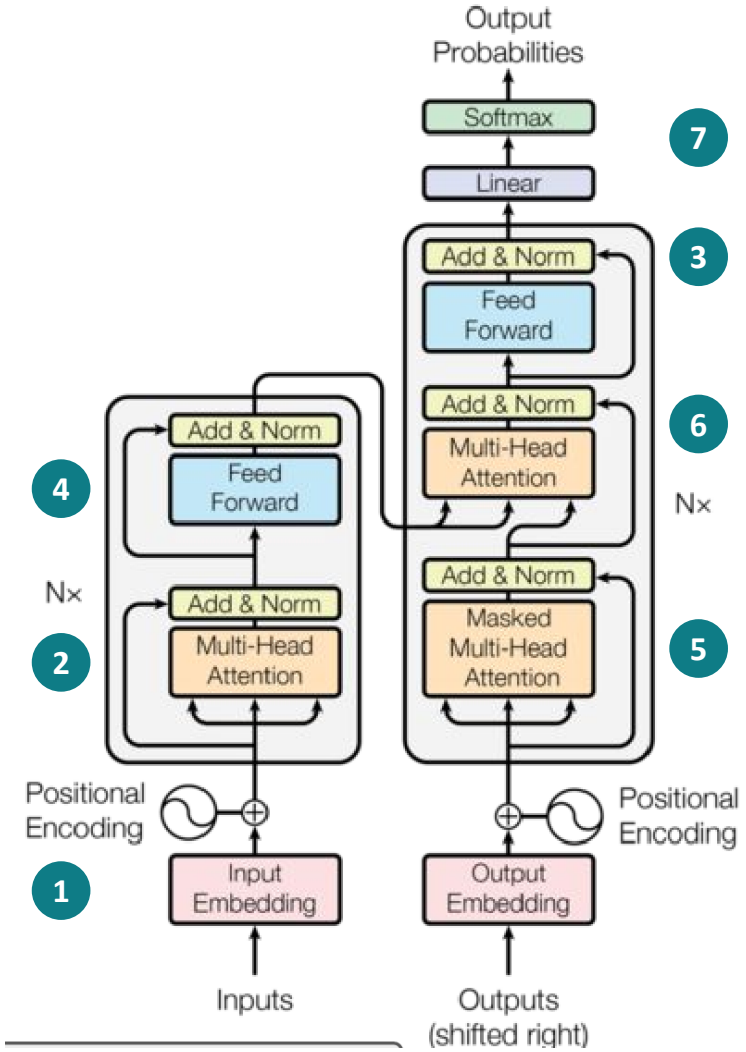
Propor o Transformer: encoder-decoder baseado só em atenção, sem RNN e sem CNN.

2 **Contribuições declaradas**

- Scaled dot-product attention e multi-head attention
- Positional encoding sinusoidal
- Estado da arte no WMT 2014: 28,4 BLEU (EN→DE) e 41,8 (EN→FR)
- Fração do custo de treino; generaliza para análise sintática

O mapa: Figura 1 do artigo

O encoder lê a frase; o decoder gera a tradução. Vamos percorrer cada bloco.



1. **Embedding + positional encoding**
   texto → vetores com posição
2. **Multi-head self-attention**
   cada token consulta todos os outros
3. **Add & Norm**
   soma residual + normalização
4. **Feed-forward**
   transforma cada posição
5. **Masked multi-head attention**
   o decoder não vê o futuro
6. **Cross-attention**
   o decoder consulta o encoder
7. **Linear + softmax**
   probabilidade do próximo token

N× = 6 camadas no encoder e 6 no decoder

Figura 1: Vaswani et al. (2017), via Agrawal, S. “Transformers” (Google), p. 26

Do texto aos tokens e aos embeddings

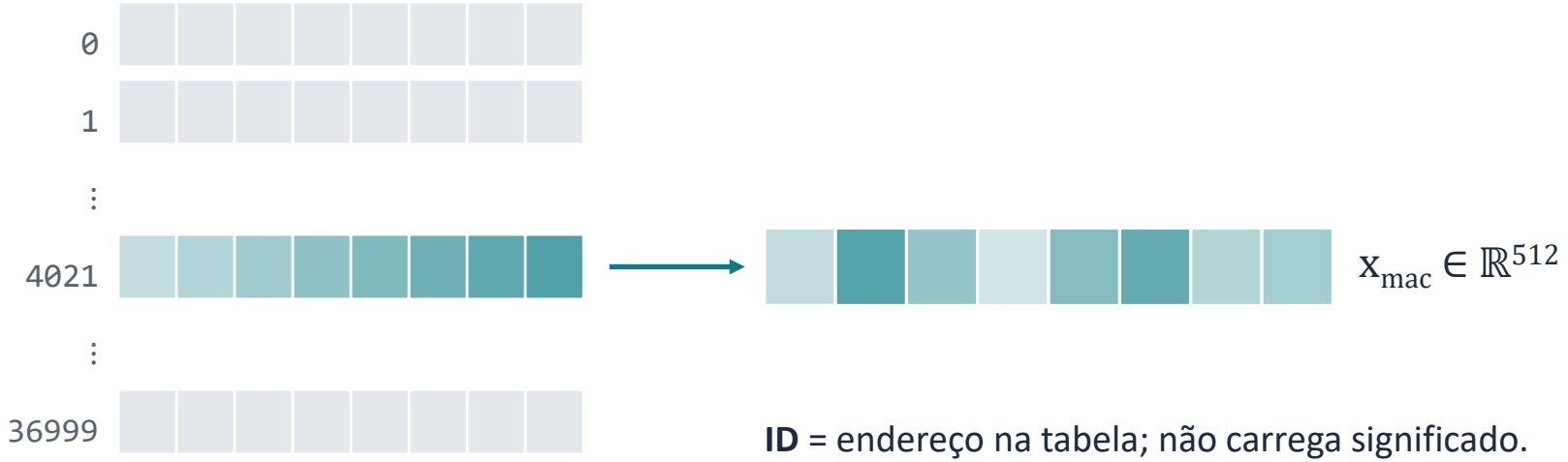
O tokenizador corta o texto; uma tabela aprendida transforma cada ID em um vetor.

“O macaco comeu a banana porque ele estava com fome.”

tokens (subpalavras)

_O	_mac	aco	_comeu	_a	_banana	_porque	_ele	…
312	4021	877	9134	57	6610	2240	1803	IDs (ilustrativos)

tabela de embeddings $|V| \times d_{model}$



ID = endereço na tabela; não carrega significado.
Embedding = vetor aprendido que representa o token; ajustado no treino.

Como representar a ordem?

Mesmos tokens, ordem diferente, significado diferente.

O **macaco** comeu a **banana**. ≠ A **banana** comeu o **macaco**.

Sem informação posicional, a self-attention do encoder não distingue explicitamente a ordem dos tokens.



conteúdo (embedding) posição (PE) entrada do encoder

$$PE_{(pos,\,2i)} = \sin(pos\ /\ 10000^{2i/d})$$

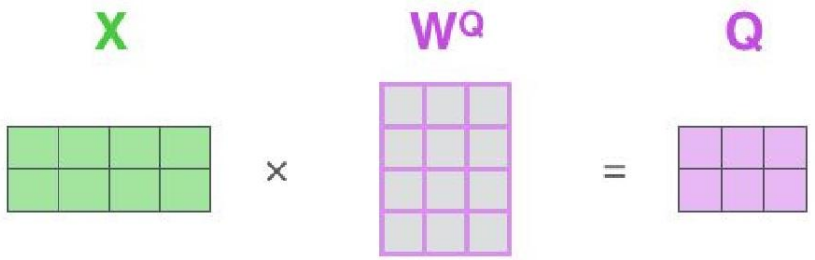
$$PE_{(pos,\,2i+1)} = \cos(pos\ /\ 10000^{2i/d})$$

d = d_{model} = 512 · cada dimensão oscila numa frequência diferente

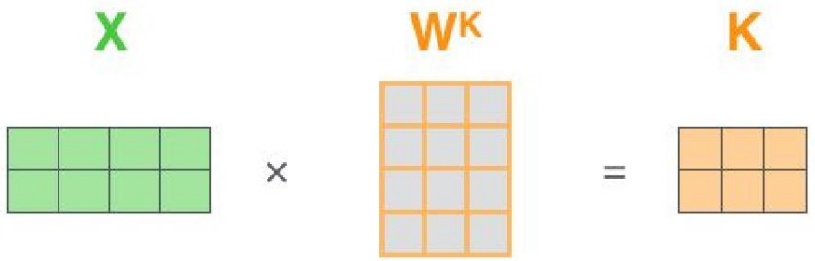
No Transformer original: senos e cossenos fixos, somados ao embedding (mesma dimensão).
Depois vieram: posições relativas e rotacionais (RoPE). Posições aprendidas, testadas no próprio artigo, deram resultado quase igual.

Query, Key e Value: três funções distintas

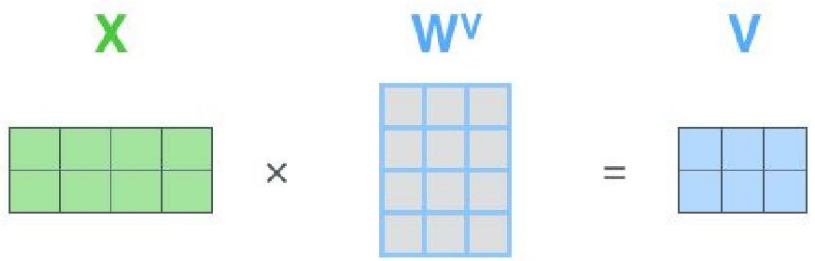
Cada token é projetado em três vetores por matrizes aprendidas.



$$Q = X\,W_Q$$



$$K = X\,W_K$$



$$V = X\,W_V$$

- **Query (Q)**: o que este token procura
- **Key (K)**: como cada token pode ser encontrado
- **Value (V)**: a informação que será combinada

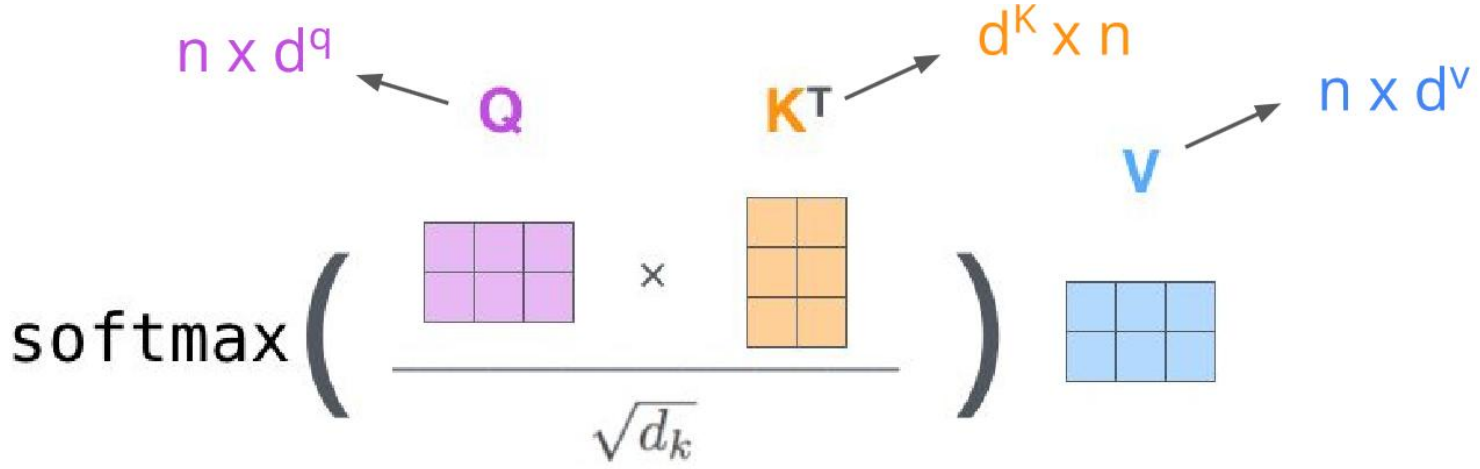
Na primeira camada, X contém embeddings com informação posicional. Nas seguintes, contém representações contextualizadas. Nesta configuração, cada matriz W_Q, W_K, W_V tem dimensão 512 × 64.

Q é comparada com K; V carrega o conteúdo.

Como se calcula a atenção

Quatro etapas: produto matricial, escalonamento, softmax e combinação ponderada

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q\,K^T}{\sqrt{d_k}}\right)V$$



A softmax distribui o peso, não escolhe um só:
softmax(2,0 | 1,0 | 0,1)
= 0,66 | 0,24 | 0,10

1 $Q\,K^T$

produto escalar: compatibilidade entre tokens (n × n)

2 $\sqrt{d_k}$

escala: evita que a softmax sature

3 softmax

por linha: pesos ≥ 0 que somam 1

4 V

média ponderada dos values ($n \times d_v$)

Exemplo: a quem “ele” presta atenção?

“O macaco comeu a banana porque ele estava com fome.” Números didáticos ($d_k = 2$), contas reais.

Dados $\qquad d_k = 2$

$$\mathbf{q}_{\text{ele}} = [2 \;\; 1] \qquad K = \begin{bmatrix} 2 & 1 \\ 1 & -2 \\ 1 & 1 \end{bmatrix} \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0{,}5 & 0{,}5 \end{bmatrix} \begin{matrix} \leftarrow \text{macaco} \\ \leftarrow \text{banana} \\ \leftarrow \text{ele} \end{matrix}$$

---

1 Produto escalar $\quad \mathbf{s} = \mathbf{q}_{\text{ele}} K^{\top} = [2 \;\; 1] \begin{bmatrix} 2 & 1 & 1 \\ 1 & -2 & 1 \end{bmatrix} = [5 \;\; 0 \;\; 3]$

2 Escala $\quad \dfrac{\mathbf{s}}{\sqrt{d_k}} = \dfrac{1}{\sqrt{2}} [5 \;\; 0 \;\; 3] \approx [3{,}54 \;\; 0{,}00 \;\; 2{,}12]$

3 Softmax $\quad \alpha_j = \dfrac{\exp(s_j/\sqrt{d_k})}{\sum_i \exp(s_i/\sqrt{d_k})}, \qquad \boldsymbol{\alpha} \approx [0{,}786 \;\; 0{,}023 \;\; 0{,}191]$

4 Soma ponderada $\quad \mathbf{z}_{\text{ele}} = \boldsymbol{\alpha} V = 0{,}786\, \mathbf{v}_{\text{mac}} + 0{,}023\, \mathbf{v}_{\text{ban}} + 0{,}191\, \mathbf{v}_{\text{ele}}$

$\approx [\mathbf{0{,}88} \;\; \mathbf{0{,}12}]$

Self-attention: linhas = quem consulta · colunas = quem é consultado

	macaco	banana	ele
macaco	0,50	0,25	0,25
banana	0,64	0,04	0,32
ele	0,79	0,02	0,19

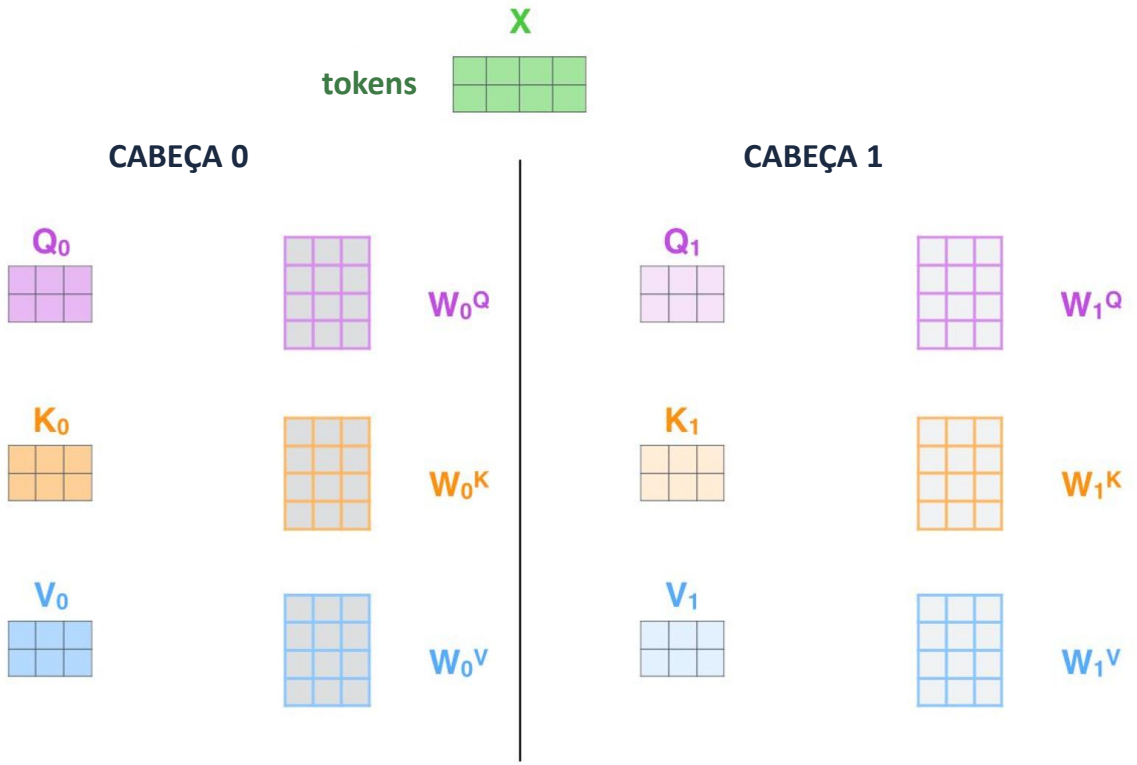
Cada linha soma 1 e cada posição recebe sua própria combinação.
Q, K e V vêm da mesma sequência.

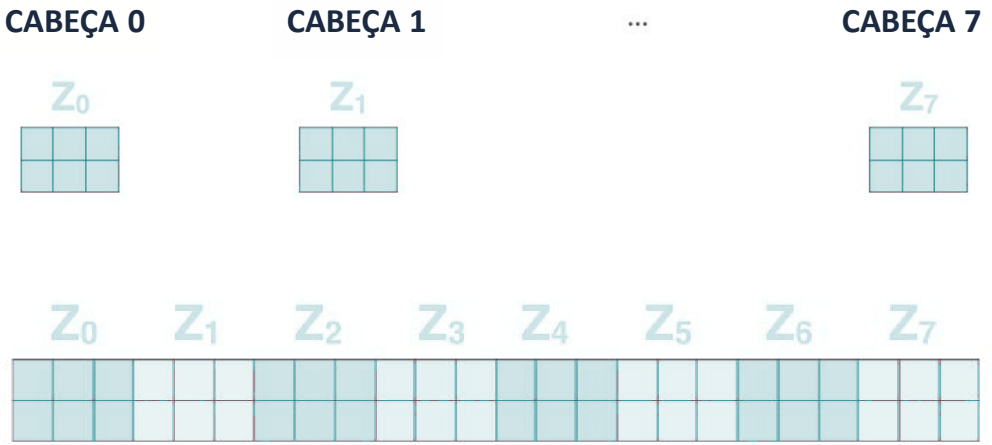
“ele” passa a carregar informação de “macaco”.

Exemplo didático construído para a apresentação (vetores escolhidos à mão; na matriz, só a linha “ele” foi calculada, as demais são ilustrativas)

Multi-head attention: várias combinações em paralelo

Cada cabeça tem projeções próprias; os resultados são concatenados.





$d^q = d^K = d^v = d/m$

m = h = número de cabeças

8 saídas Z concatenadas → × W^O

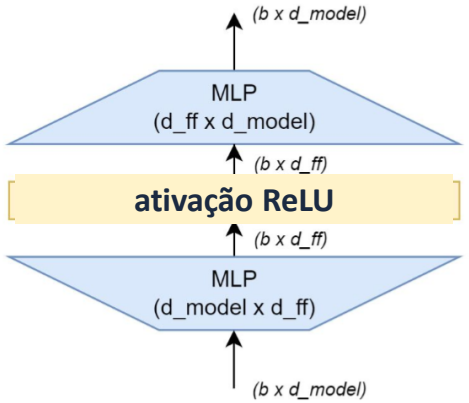
$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \qquad h = 8,\ d_k = 512/8 = 64$$

O que cada cabeça aprende emerge do treino; não há uma função linguística fixa por cabeça.

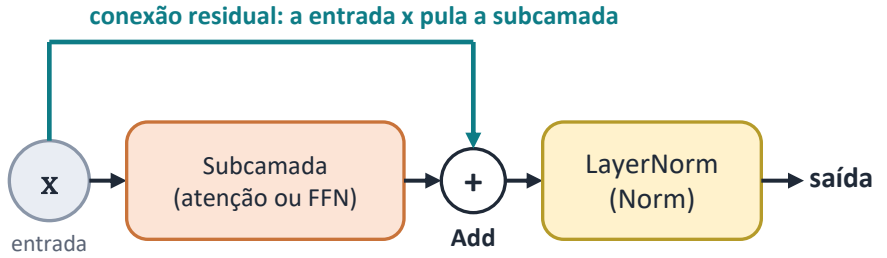
Feed-forward, Add & Norm e o encoder completo

A atenção troca informação entre tokens; a feed-forward transforma cada token.



$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

$$d_ff = 4 \times d$$



$$\text{saída} = \mathrm{LayerNorm}(x + \mathrm{Subcamada}(x))$$

Feed-forward: expande (512 → 2048), aplica ReLU e reduz (→ 512); mesma rede em cada posição.

Add & Norm: soma a entrada (residual) e normaliza.

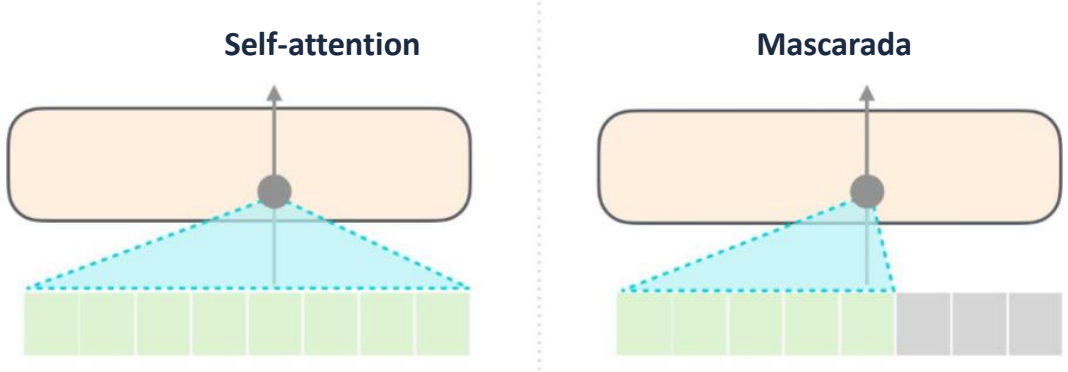
Encoder = [self-attention → Add & Norm → FFN → Add & Norm] × 6

Mesma estrutura em cada camada, mas **parâmetros diferentes**.
Saída: um vetor contextualizado por token (n × 512), que segue para o decoder.

Original: LayerNorm(x + Subcamada(x)) (Post-LN). Muitas variantes atuais usam Pre-LN.

O decoder e a máscara causal

Ao prever o próximo token, cada posição consulta a si mesma e as posições anteriores.



O decoder gera “The monkey ate…” um token por vez.

A máscara põe −∞ nas posições futuras **antes** da softmax → peso 0.

Ex.:
“monkey” consulta <início>,
The e monkey, mas não “ate”.

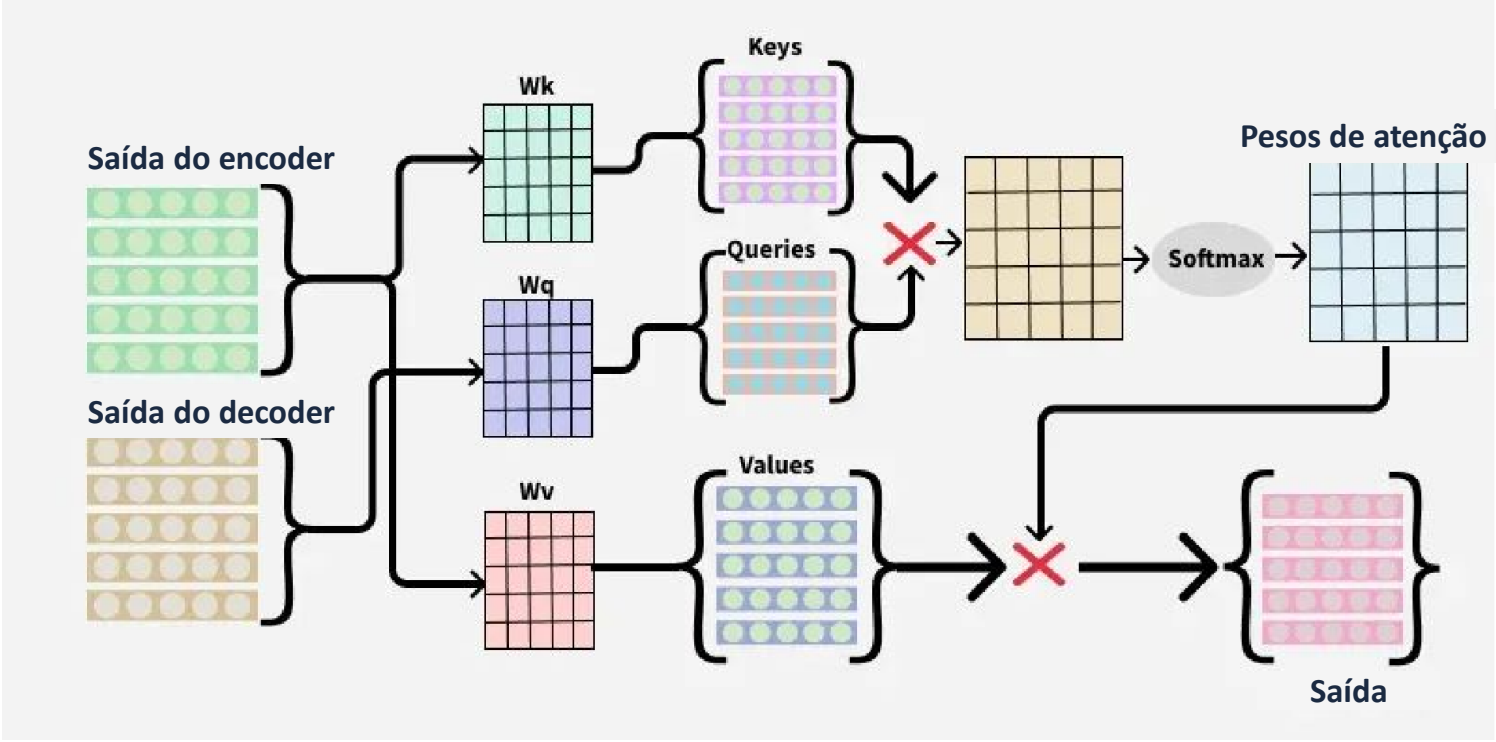
	<início>	The	monkey	ate
<início>	s_{ij}	$-\infty$	$-\infty$	$-\infty$
The	s_{ij}	s_{ij}	$-\infty$	$-\infty$
monkey	s_{ij}	s_{ij}	s_{ij}	$-\infty$
ate	s_{ij}	s_{ij}	s_{ij}	s_{ij}

$$\mathrm{softmax}(QK^T/\sqrt{d_k} + M), \quad M_{ij} = -\infty \text{ se } j > i$$

linhas: quem consulta · colunas: quem é consultado

Cross-attention: conectar encoder e decoder

Q vem da representação intermediária do decoder. K e V vêm da saída do encoder.



Ao gerar “monkey”, a query do decoder procura “macaco” na frase de origem.

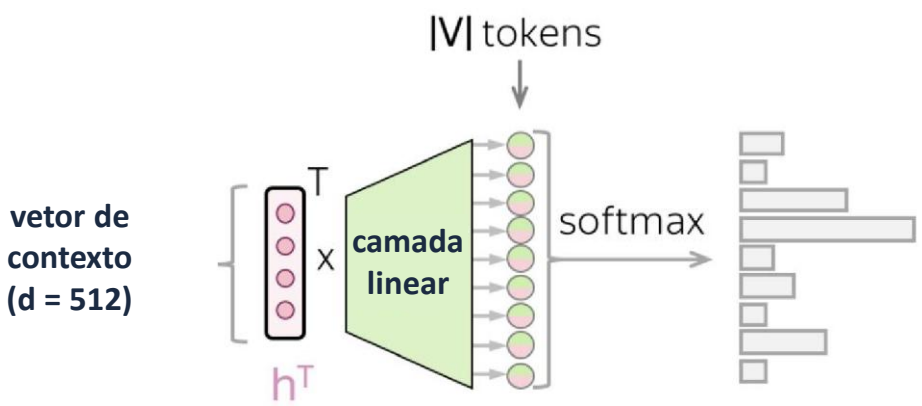
Self-attention: Q, K e V vêm da mesma sequência.

Cross-attention: Q vem do decoder; K e V vêm do encoder.

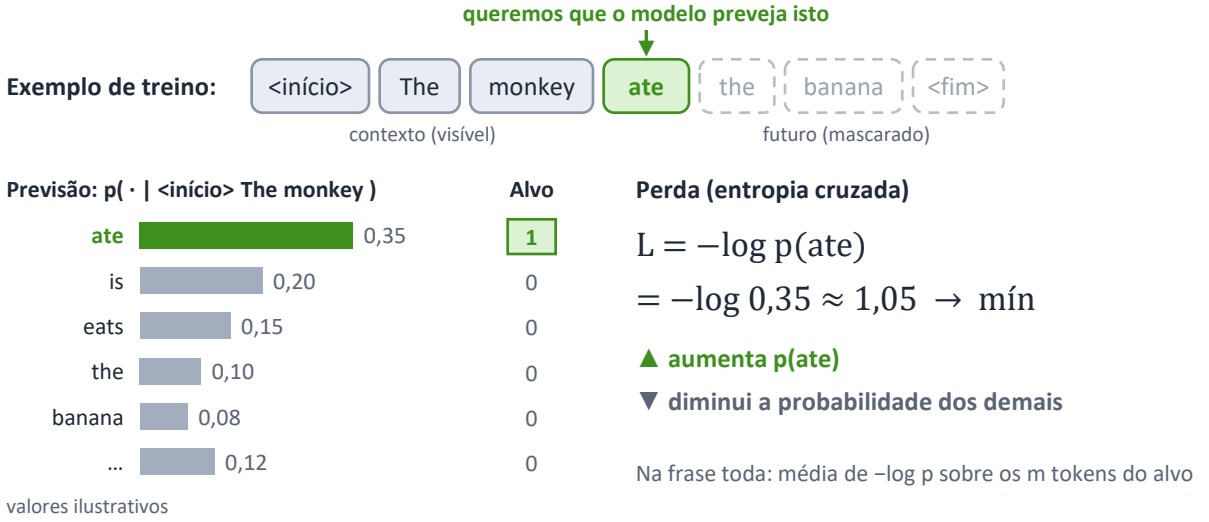
Mesma fórmula: Attention(Q_{dec}, K_{enc}, V_{enc}). Cumpre o papel da atenção de Bahdanau, mas com produto escalar e sem RNN.

Da representação ao próximo token, e como o modelo aprende

Linear + softmax sobre o vocabulário; a perda compara a previsão com o token correto.



Softmax **de saída**: distribuição sobre |V| ≈ 37 mil tokens na configuração EN→DE do artigo (diferente da softmax **dentro** da atenção).



Previsão → **Perda (entropia cruzada)** → **Gradientes (backprop)** → **Atualização (Adam)**

Entrada deslocada: <início> The monkey … → alvo: The monkey ate …

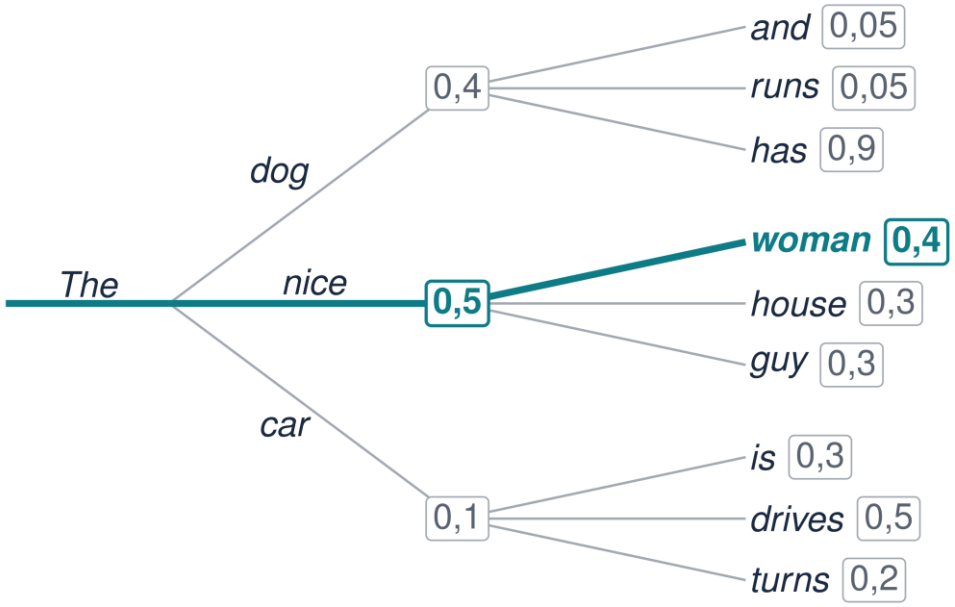
Aprendidos no treino: W_Q, W_K, W_V, W^O, FFN, embeddings. **Calculados** a cada entrada: os pesos de atenção.

Como o modelo gera uma sequência

Um token por vez: cada token gerado volta como entrada do decoder.

Greedy (guloso)

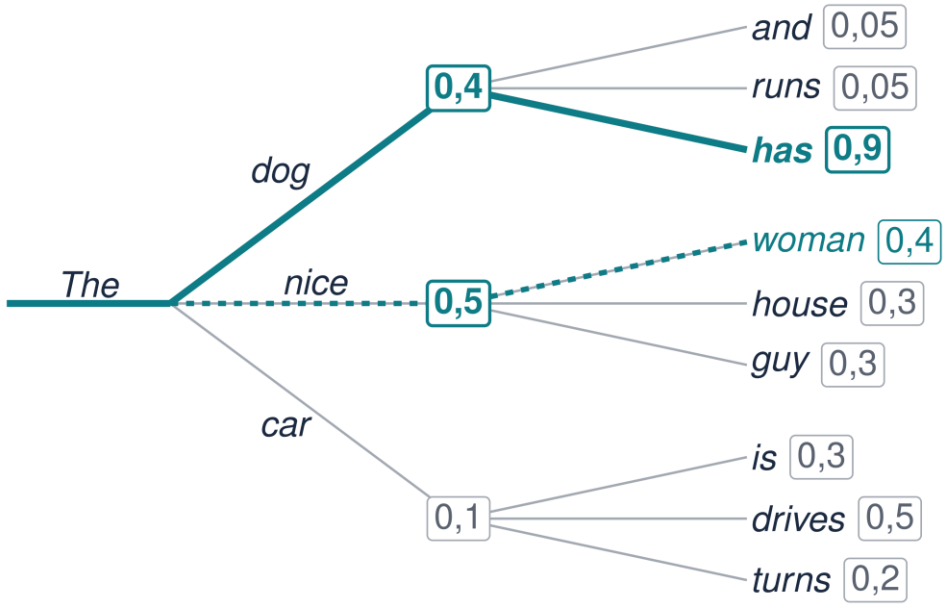
The nice woman: 0,5 × 0,4 = 0,20



Escolhe sempre o token mais provável em cada passo.

Beam search (k = 2)

The dog has: 0,4 × 0,9 = 0,36



Mantém as k melhores sequências parciais.

Geração token a token: **<início> → The → monkey → ate → … → <fim>**

Termina ao gerar <fim> ou ao atingir o comprimento máximo (entrada + 50 tokens). **No artigo:** beam search com k = 4 e α = 0,6.
Amostragem / temperatura: comum em LLMs; o artigo não usa.

RESULTADOS · TABELA 2

Dados e resultados: Tabela 2 do artigo

O Transformer big alcançou resultados de referência em tradução, com menor custo de treino que vários sistemas anteriores

Modelo	BLEU		Custo de treino (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

WMT 2014 EN→DE
4,5 M pares de frases

WMT 2014 EN→FR
36 M pares de frases

8 GPUs P100
base: 12 h · big: 3,5 dias

+2,0 BLEU
sobre o melhor ensemble anterior (EN→DE)

Funcionou: estado da arte em EN→DE (28,4) e EN→FR (41,8); o modelo base é o mais barato da tabela ($3{,}3 \cdot 10^{18}$ FLOPs).

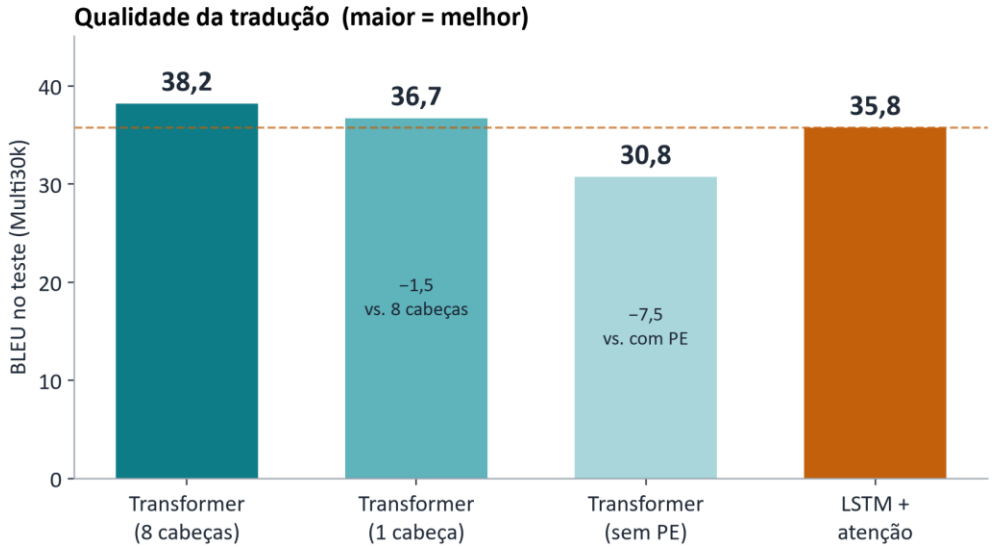
Ressalva: o modelo base fica abaixo dos anteriores em EN→FR (38,1). Teste: newstest2014 · métrica: BLEU.

RESULTADOS

Reprodução: o Transformer supera o LSTM

Multi30k alemão → inglês (29 mil pares) · Google Colab, GPU T4 · mesma tokenização BPE e 15 épocas para todos

Medida	Artigo (WMT14)	Reprodução
BLEU Transformer (8 cabeças)	27,3	**38,2**
BLEU LSTM + atenção	24,6 (GNMT+RL)	35,8
Tempo de treino	12 h no total · 8 GPUs	**21,9 s** vs. 58,0 s
Parâmetros	65 M	**7,6 M** vs. 27,1 M
1 cabeça vs. 8 cabeças	−0,9 BLEU	**−1,5 BLEU**
Sem positional encoding	não testado	**−7,5 BLEU**



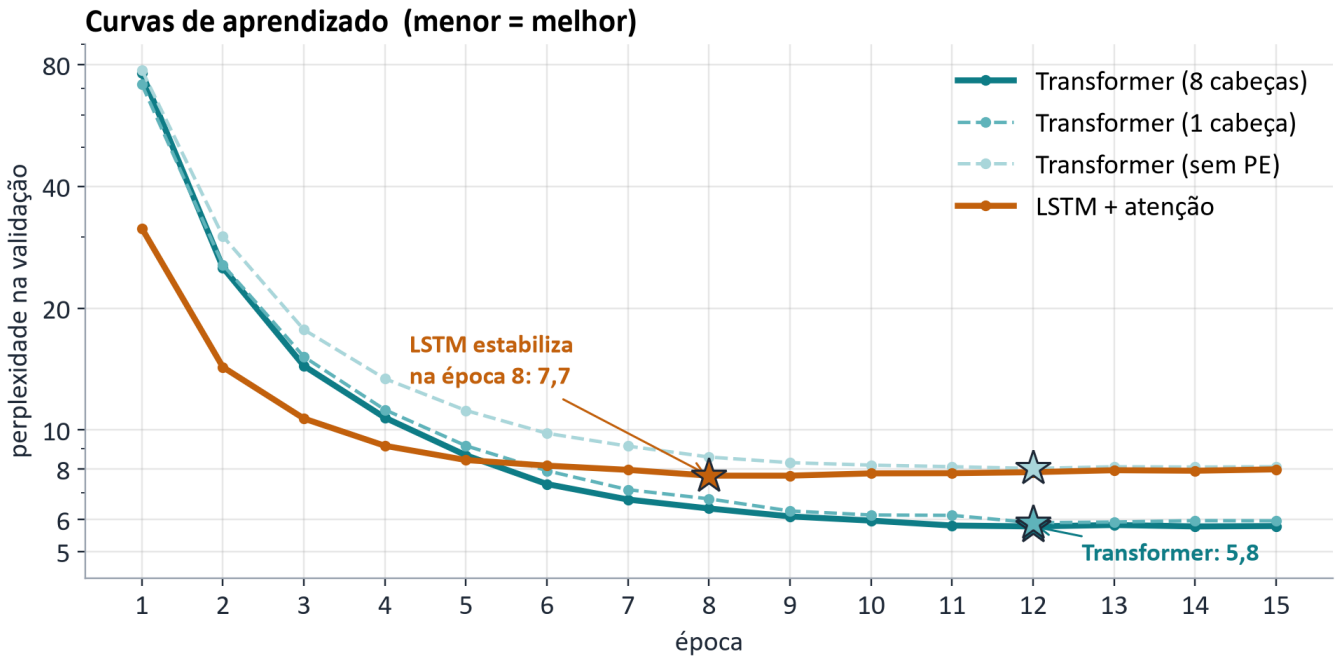
Tendências reproduzidas: Nesta execução, o Transformer obteve maior BLEU, com menor tempo por época e menos parâmetros que o LSTM avaliado; múltiplas cabeças e o positional encoding ajudaram.

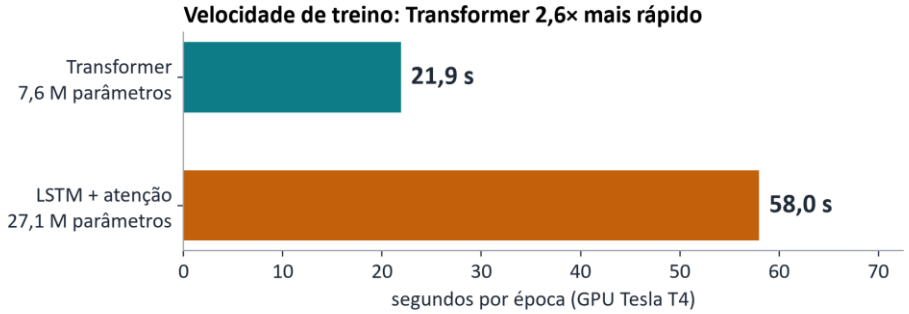
Multi30k ≠ WMT14: os valores absolutos de BLEU não são comparáveis com os do artigo. Uma única execução (seed 42), decodificação greedy, modelo pequeno (N = 3, d_model = 256).

RESULTADOS

Por que o Transformer vence: velocidade e aprendizado

Mesmos dados e mesma GPU: o Transformer treina mais rápido e continua aprendendo por mais tempo.





- **2,6× mais rápido por época:** contribuem o paralelismo das posições e o modelo menor; o LSTM avança palavra por palavra.
- **LSTM estabiliza na época 8** (perplexidade 7,7); o Transformer cruza na época 5 e chega a 5,8.

estrela = melhor época (modelo avaliado no teste) · perplexidade ≈ entre quantas palavras o modelo hesita

Exemplo do teste: “Leute reparieren das Dach eines Hauses.” → Transformer: “people are fixing the roof of a house.” (igual à referência) · LSTM: “people repairing the roof of a house.”

Limite: “Der Affe isst die Banane, weil er hungrig ist.” → “the monkey is eating the bananas who is also is weigased.” Acerta quem come o quê, mas erra o final da frase.no Multi30k (Affe, hungrig).

ANÁLISE CRÍTICA DO TRABALHO

Pontos fortes, limitações e melhorias

A atenção total explica tanto a força quanto o principal limite.

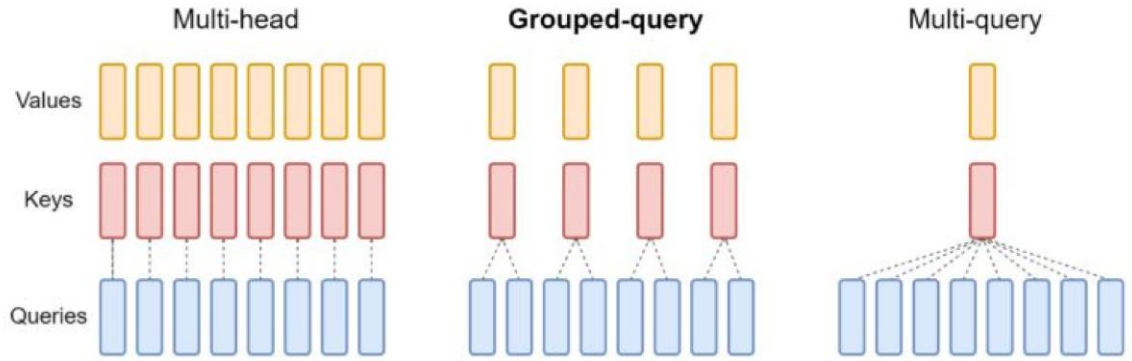
Pontos fortes

- Processamento paralelo das posições durante o treino
- No encoder, cada token pode consultar todos os tokens válidos
- Bloco genérico: base de BERT, GPT, T5 e ViT

Limitações

- Custo $O(n^2 \cdot d)$: cresce com o quadrado do comprimento
- A avaliação da tradução enfatiza BLEU e não inclui avaliação humana
- Geração ainda token a token; o cache de K/V cresce

Aplicações práticas: tradução, LLMs (GPT), busca (BERT), visão (ViT), fala (Whisper).



Melhoria: compartilhar K/V entre cabeças (multi-query / grouped-query) reduz a memória na inferência.

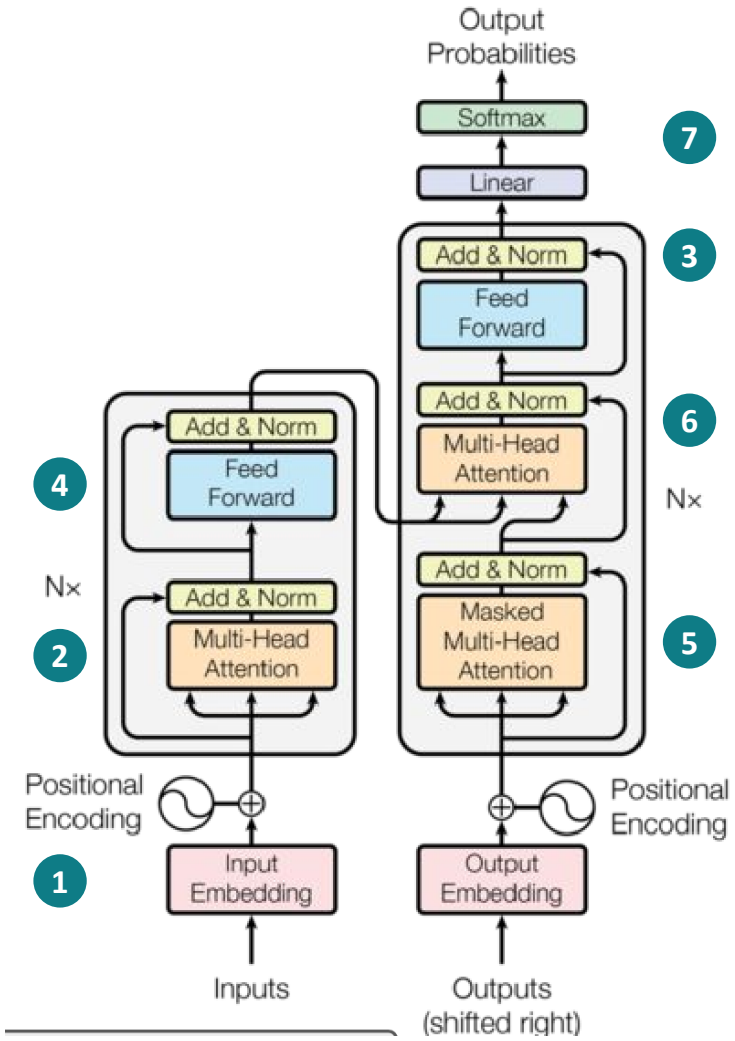
Outras melhorias

- Atenção eficiente para textos longos (esparsa, FlashAttention)
- Posição relativa ou rotacional (RoPE)
- Pre-LN para treinar modelos profundos com estabilidade

CONCLUSÃO

O percurso completo

Do texto à tradução, e três perguntas para checar o entendimento.



- (1) Tokens → embeddings + posição
- (2) Self-attention: cada token consulta os demais
- (3–4) Add & Norm + FFN, × 6 → vetores contextualizados
- (5) Decoder mascarado consulta o próprio prefixo
- (6) Cross-attention consulta o encoder
- (7) Linear + softmax → próximo token

Treino: alvo conhecido, todas as posições em paralelo. **Inferência:** um token por vez.

PARA CHECAR

1. Qual a diferença entre os parâmetros W_Q e os pesos de atenção?
2. Por que a máscara é aplicada antes da softmax?
3. De onde vêm Q, K e V na cross-attention?

Referências

Vaswani, A. et al. **Attention Is All You Need.** NeurIPS, 2017. arXiv:1706.03762

Agrawal, S. **Transformers** (slides, Google). Material de apoio, figuras e créditos indicados em cada slide.

Alammar, J. **The Illustrated Transformer.** 2018. jalammar.github.io/illustrated-transformer

Rush, A. (2018); Huang, A. et al. (2022). **The Annotated Transformer.** Harvard NLP.

Zhang, A. et al. **Dive into Deep Learning**, cap. 11. d2l.ai

Hugging Face. **Curso de NLP**, cap. 1.4 (pt). huggingface.co/docs/course/pt

von Platen, P. **Transformers-based Encoder-Decoder Models.** Hugging Face Blog, 2020.

Roitman, H. **The Hitchhiker's Guide to Agentic AI.** arXiv:2606.24937, 2026.

ENCERRAMENTO · QUIZ

Quiz

Escaneie o código com o celular ou acesse o link.



Link do formulário

https://forms.gle/8gPrqnk9qUkxrnb28

ENCERRAMENTO

Obrigado!

Perguntas e comentários são bem-vindos.

Bryan Fernando Sarango Rodriguez
bryan.rodriguez@dtel.Inatel.br

Inatel